

# End-to-End SRE Practices in a Multi-Orchestrated Microservices System

Date: 2026-05-19

Repository: [https://github.com/Tedra-ez/IntroductionSRE\\_End](https://github.com/Tedra-ez/IntroductionSRE_End)

Stack: Docker Compose / Swarm, Kubernetes, Terraform, Ansible

Prometheus <http://localhost:9091> | Grafana <http://localhost:3001> (admin/admin)

## 1. Abstract

This project implements Site Reliability Engineering across a distributed shop with six microservices, API gateway, Nginx frontend, PostgreSQL, and Redis. Observability uses Prometheus and Grafana with SLO-based alerts. A simulated order-service database incident was detected, mitigated, and documented in a postmortem.

## 2. Objectives Met

- 6+ microservices with health checks and /metrics
- Docker Compose and Docker Swarm (docker-stack.yml)
- Kubernetes manifests with HPA for order-service
- Terraform inventory provisioning (extensible to cloud VMs)
- Ansible site playbook for Docker and stack deployment
- SLIs/SLOs: 99% availability, 200ms P95, 1% error rate
- Prometheus + Grafana + alert rules
- Incident simulation and postmortem
- Capacity planning document

## 3. Architecture

User -> Frontend (Nginx :8088) -> API Gateway -> Auth, Product, Order, Payment, Notification, User Profile  
-> PostgreSQL / Redis. Monitoring: Prometheus :9091 -> Grafana :3001.

## 4. Microservices

auth-service, product-service, order-service, payment-service, notification-service, user-profile-service (+ api-gateway, frontend).

## 5. SLI / SLO

SLIs: availability, latency, error rate, success rate. SLOs: availability  $\geq 99\%$ , P95 latency  $\leq 200\text{ms}$ , error rate  $\leq 1\%$ . Rules: monitoring/prometheus/alerts.yml. Details: docs/SLI\_SLO.md.

## 6. Prometheus - Targets and Metrics

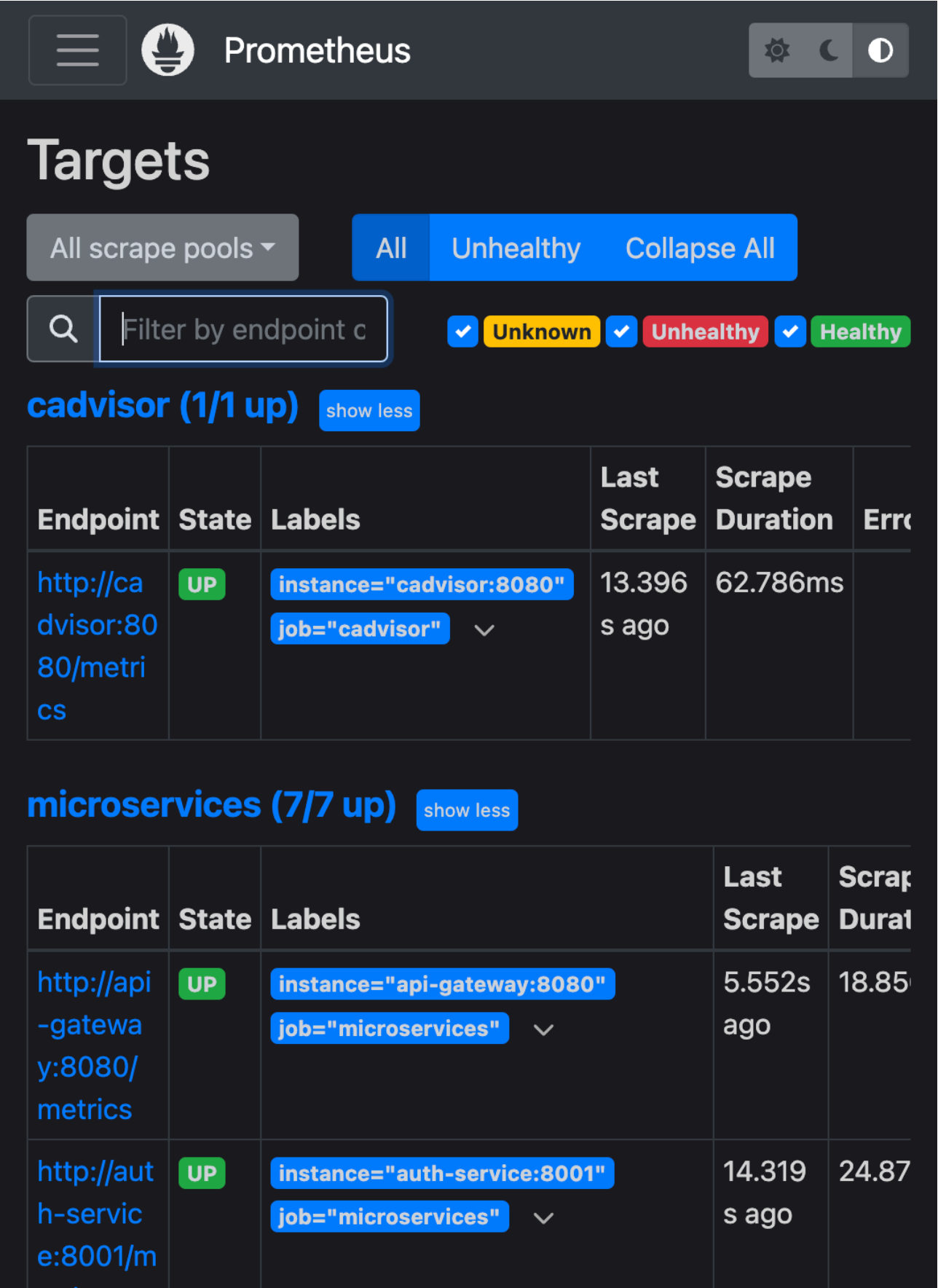


Figure 1: Prometheus targets (microservices UP)

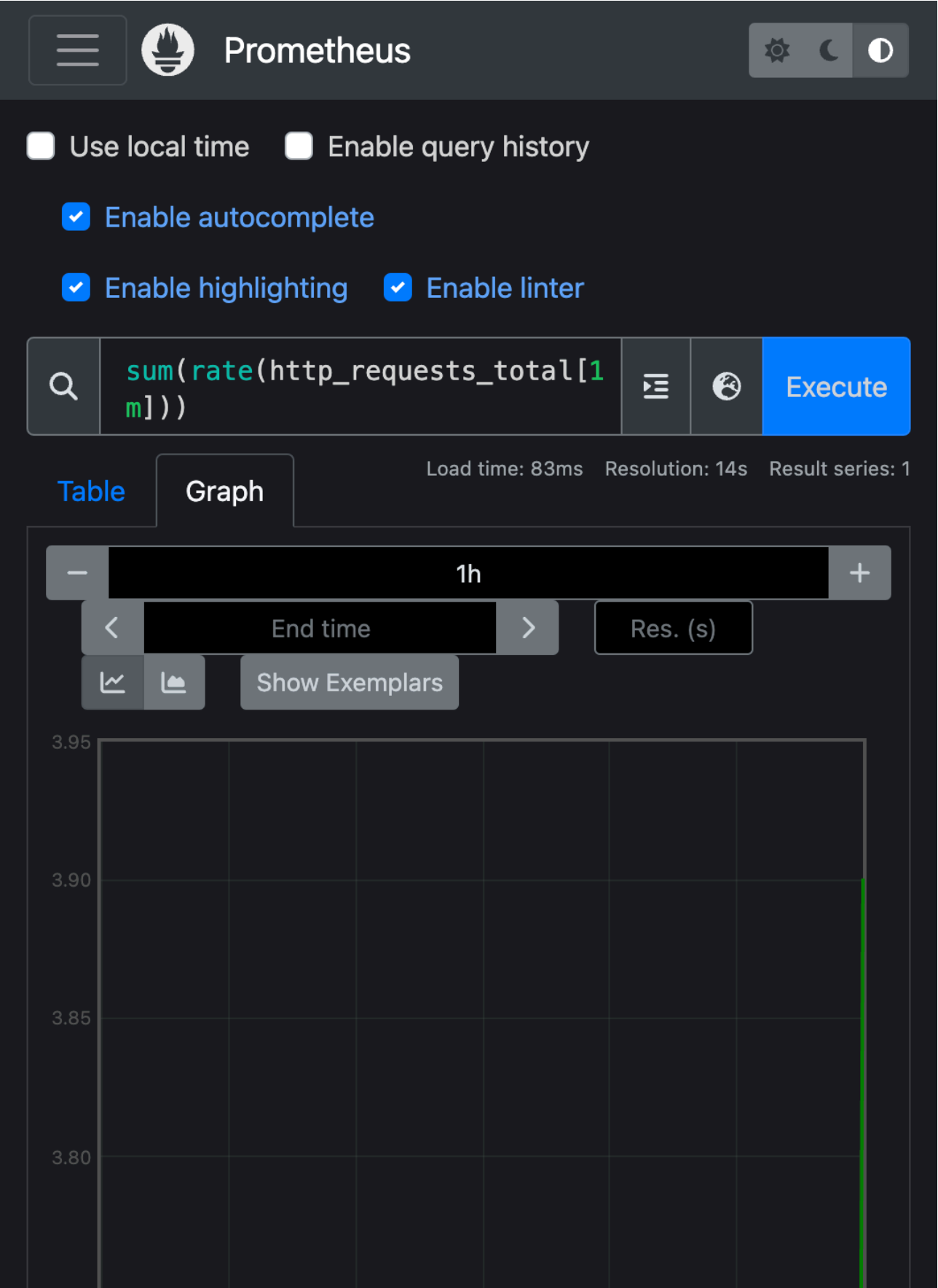


Figure 2: Prometheus query - http\_requests\_total rate

7. Grafana Dashboard

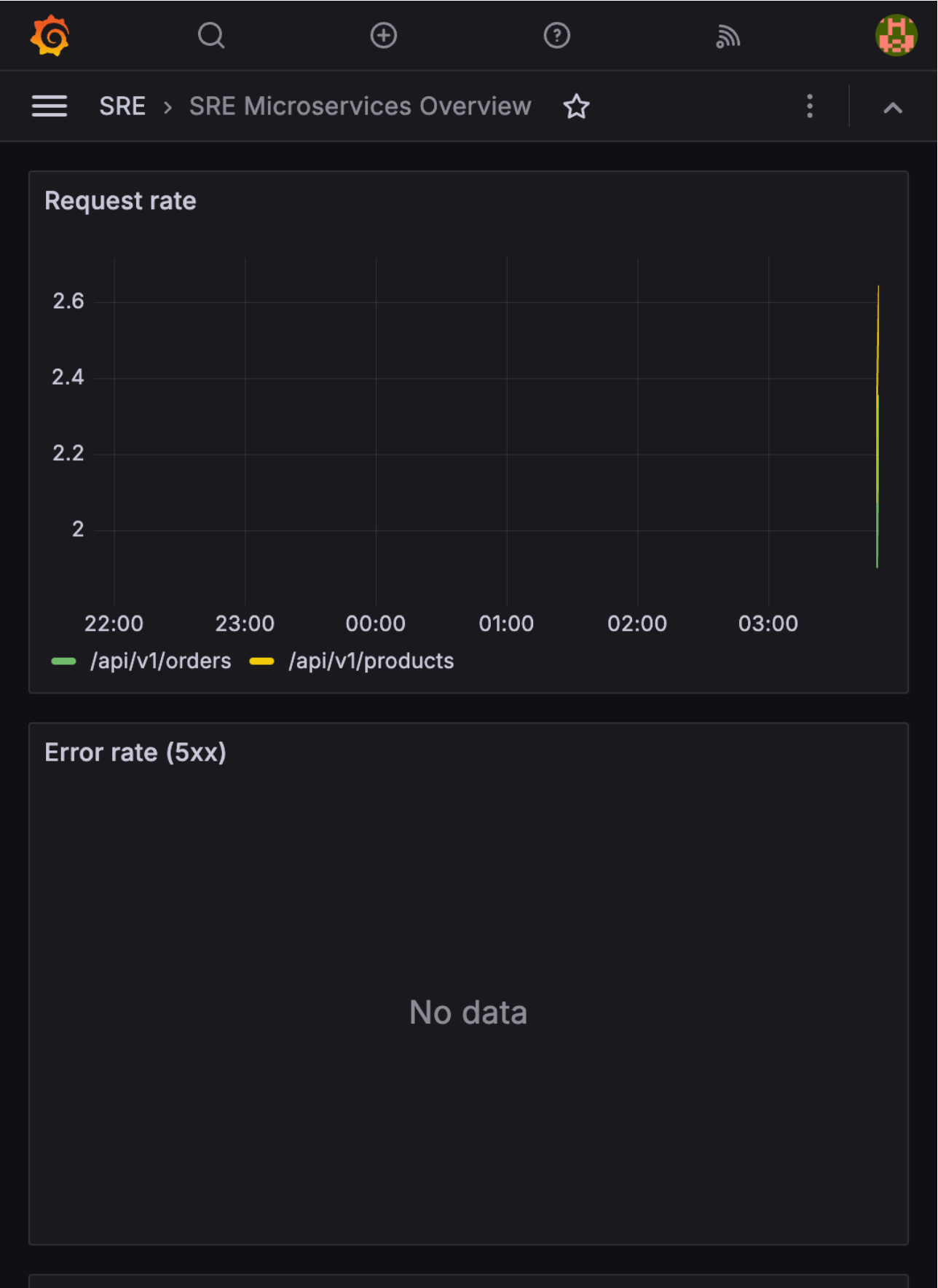


Figure 3: SRE Microservices Overview dashboard

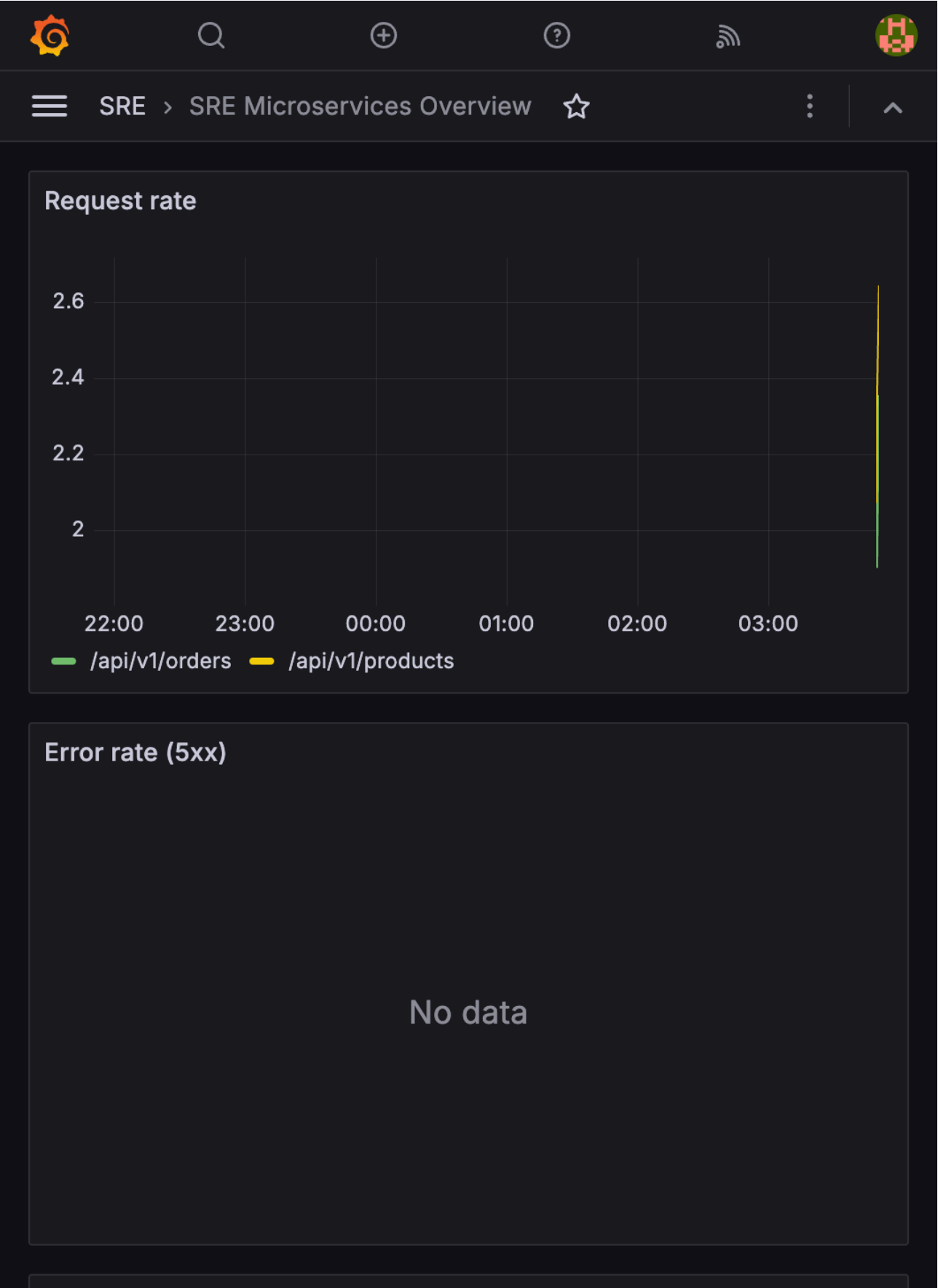


Figure 4: Grafana - error rate and latency panels

8. Incident Response (Assignment 4)

Scenario: order-service failed due to incorrect PostgreSQL password. Detection: OrderServiceDown alert and

## **End Term Project - Comprehensive SRE Implementation**

Grafana error spike. Recovery: restore env vars, recreate container. Full timeline: docs/postmortem.md.

### **9. Infrastructure as Code**

Terraform: terraform/ generates Ansible inventory. Ansible: ansible/site.yml installs Docker and deploys compose stack.

### **10. Multi-Orchestration**

Swarm: docker swarm init && docker stack deploy -c docker-stack.yml sre. Kubernetes: kubectl apply -f k8s/ (namespace, deployments, services, HPA).

### **11. Capacity Planning**

Order and payment services dominate CPU under load; PostgreSQL is the primary bottleneck. Mitigations: horizontal replicas, vertical DB scaling, async notifications. See docs/capacity-planning.md.

### **12. Conclusion**

The system demonstrates the full SRE lifecycle: design, deploy, monitor, alert, respond, automate, and plan capacity across Swarm and Kubernetes.

## **13. Deliverables and Git Repository**

### **Source code and IaC:**

<https://github.com/your-repo>

Submit this PDF only, with the Git link above. Clone the repo and run: `docker compose up -d --build`